Introduction

Prime numbers are fundamental to modern cryptography. Secure communications through encryption is often achieved through generation of very large prime numbers. For example, RSA encryption relies on generating keys by multiplying two large prime numbers. Therefore, breaking RSA encryption requires integer factorization techniques, which is known to be a computationally difficult problem, provided the prime numbers used are sufficiently large. Efficient computation of large prime numbers is needed for cryptography.

One of the issues with finding prime numbers is their relative sparsity as the size of the prime numbers increases. According to the Prime Number Theorem, the probability that a random integer N is prime is close to $\frac{1}{log(N)}$. This means for numbers with 1000 digits, approximately 1 in 2302 are prime. Primality algorithms need to test thousands of numbers in order to find just one prime, and therefore need to be very well optimized.

One of the difficulties with primality algorithms is uncertainty in their validity and uncertainty in asymptotic runtime. For some algorithms, it is unknown whether the test is deterministic (i.e. it will always correctly determine that a number is prime), such as the Baillie-PSW primality test. In some cases, the asymptotic runtime of algorithms is contingent on conjectures in number theory. For example, the AKS primality test has a runtime of $O(log^6(n))$ unconditionally, but this can be reduced to $O(log^3(n))$ if Agrawal's Conjecture is true. Some tests take probabilistic approaches to speed up runtime. Typically, probabilistic approaches will only prove a number is composite, and repeatedly executing the test will increase certainty that a number is prime, such as the Miller-Rabin test. Additionally, some algorithms have high constants, which asymptotic analysis fails to capture. For these reasons, a simple asymptotic runtime analysis is insufficient to fully understand the differences between algorithms.

Furthermore, there are algorithms that determine whether a range of numbers are prime, rather than a single number. The Sieve of Eratosthenes can be used to find all prime numbers between 1 and n with a time complexity of $O(n\, log^2(n))$ and a space complexity of $O(n)$. Sieve methods are typically very effective for finding a large number of small primes, but have high space complexity, making them impractical for finding large prime numbers used in cryptography. Therefore, we will not be discussing sieve methods further in this project.

Efficient primality algorithms, regardless of determinism, can achieve a time complexity of $O(log^c(n))$, where $c < 10$ is a constant. The Miller-Rabin test is the most efficient algorithm that we analyze, which achieves $O(k\, log^3(n))$, where k is the number of iterations.

Approach

This project explores three distinct approaches to determining the primality of a given number. The first of these is the trial division primality test. This approach aims to determine if a number *N* is prime by directly checking all possible factors of *N*. For each possible factor *i* (between *2* and *N-1*), the algorithm would check if *N* is evenly divisible by *i*. If this is the case,

then *N* must be composite. If it is found that *N* is not evenly divisible by any of its potential factors, the algorithm can conclude that *N* is prime.

There are ways in which this approach can be optimized to lessen the number of factors checked. Using the fact that no number greater than *N/2* in the range *(2, N - 1)* can evenly divide *N*, the algorithm need only iterate from *2* to *N/2*. Furthermore, it is true that any factor of *N* is greater than or equal to $\sqrt{N}$, its corresponding factor must be less than or equal to $\sqrt{N}$. The trial division algorithm can utilize this to further limit the number of potential factors to check to be from *2* to $\sqrt{N}$. This will still cover all possible factors of *N* to provide an accurate primality label on *N*.

The second approach is the Fermat primality test. This approach uses Fermat's Little Theorem to make assumptions about the primality of a given number *N*. Fermat's Little Theorem states that for any prime number p and integer a, it holds that $a^p \equiv a \ mod\, p$. If we assume $a \leq p - 1$, we can divide by a on both sides to get $a^{p-1} \equiv 1 \ mod\, p$. If p is composite, this generally does not hold. The overall approach aims to the check the primality of p by checking that for any potential factor of p, the expression derived from Fermat's Little Theorem holds.

Inspecting all possible factors leads to a greater runtime than the previous approach. However, because Fermat's Little Theorem is more informative than simply checking if p is divisible by a, the algorithm can be more sure of a given number's primality without checking all possible factors. Consequently, this approach only checks a predefined integer k number of potential factors. If the theorem does not hold for any one of the k tests, the algorithm can definitively conclude that the given number is composite. Otherwise, if the theorem holds for all k tests, the algorithm will conclude that the given number is probably prime. For each test, the algorithm will randomly select a potential factor a between 2 and k - 2 to conduct the test on.

The third approach is the Miller-Rabin primality test. This builds on the previous test by not only checking that the given number p and randomly selected potential factor a satisfy Fermat's Little Theorem, but also by examining the square roots of a^{p-1}, which can be seen as the factors of a decomposition of the number in the image above. If $a^{p-1} \ mod\, p$ does not evaluate to 1, the test can immediately determine that p is composite. However, if this expression does evaluate to 1, p is not guaranteed to be prime, so the test further inspects p by dividing the exponent ($p - 1$) by 2, or taking the square root of a^{p-1}. If p is truly prime, computing $a^{(p-1)/2} \ mod\, p$ will evaluate to either 1 or $p - 1$. As the exponent continues to get divided by 2 (while the exponent remains an integer), $a^{(p-1)/2^s} \ mod\, p$ will not yield a number outside of 1 or $p - 1$ unless $a^{(p-1)/2^{s-1}} \ mod\, p$ evaluates to $p - 1$, or if p is composite. The total number of times the exponent can be evenly divided by 2 is determined by a positive integer s, where $p - 1 = 2^s \cdot d$ and d is an odd integer.

The Miller-Rabin primality test utilizes this to continuously double the exponent (starting from an exponent of d) for a number i<s times until the expression $a^{2^i \cdot d} \ mod\, p$ evaluates to $p - 1$, indicating p is likely to be prime, or until the exponent reaches $2^s \cdot d$ and the expression has not yet evaluated to $p - 1$, indicating that the number p is definitely composite. This extra layer of

checking on top of the Fermat primality test further reduces the probability of incorrectly labeling a composite number as prime. The Miller-Rabin test still checks a predefined number k of randomly generated potential factors, using Fermat's Little Theorem and the square root check on each to ultimately determine if the provided input is prime.

Figure 1. Expansion of Fermat's Little Theorem

$$
\begin{aligned}
a^{n-1} \equiv 1 \bmod n \quad &\iff a^{2^s d} - 1 \equiv 0 \bmod n \\
&\iff (a^{2^{s-1}d} + 1)(a^{2^{s-1}d} - 1) \equiv 0 \bmod n \\
&\iff (a^{2^{s-1}d} + 1)(a^{2^{s-2}d} + 1)(a^{2^{s-2}d} - 1) \equiv 0 \bmod n \\
&\quad \vdots \\
&\iff (a^{2^{s-1}d} + 1)(a^{2^{s-2}d} + 1) \cdots (a^{d} + 1)(a^{d} - 1) \equiv 0 \bmod n
\end{aligned}
$$

Implementation

The implementation of trial division primality takes in an input N and iterates through all integers i from 2 to $\lfloor \sqrt{N} \rfloor$. For each i, if N modulo i is equal to 0, indicating that N is evenly divisible by i, the algorithm terminates and returns false. Otherwise, if the program has not terminated after checking all possible values of i, the algorithm returns true after the for-loop, indicating that N is prime.

Figure 2. Pseudocode of Trial Division

```
function isPrimeNaive(n)
    i ← 2
    while i * i ≤ n do
        if n ≡ 0 (mod i) then
            return False
        end if
        i ← i + 1
    end while
    return True
end function
```

The implementation of the Fermat primality test takes in an input N and uses a for-loop to iterate $k = 5$ times. For each iteration, the algorithm generates a random number a between 2 and $N - 1$ inclusive. The algorithm then checks if Fermat's Little Theorem holds by checking if $a^{p-1} \ mod \ p = 1$. If this is not the case, the program terminates returns and false to indicate that N is composite. Otherwise, the algorithm continues on to the next iteration in the for-loop. If after k iterations through the for-loop the program has not terminated, the program returns true to indicate that N is probably prime.

Figure 3. Pseudocode of Fermat Primality Test

```
function isPrimeFermat(n)
    i ← 0
    while i < 5 do
        a ← rand(2, n − 1) # random number between 2 and n-1 inclusive
        if a^(n−1) ≡ 1 (mod n) then
            return False
        end if
        i ← i + 1
    end while
    return True
end function
```

The implementation of the Miller-Rabin primality test relies on a helper function to determine if a number is definitely composite. The function takes in a number N to determine the primality of, a as a potential factor of N, d as the initial exponent to test, and s as the number of times d can be doubled.

The function initializes the number x with the definition $a^d \ mod \ N$. If this evaluates to either 1 or $N - 1$, the function immediately returns false, indicating that N may be prime. Otherwise, the function iterates through a for-loop up to s times. Within this for-loop, the function takes the square of x to compute $x^2 \ mod \ N$ and saves the computed value in the variable x. If this new value of x is equal to $N - 1$, the function terminates early and returns false. If for all s iterations the newly computed value of x is not equal to $N - 1$, the function returns true, indicating that N must be composite.

Figure 4. Pseudocode of Subroutine to Check Compositeness

```
function checkComposite(n)
    x ← a^d mod n
    if x == 1 || x == n − 1 then
        return False
    end if
    r ← 1
    while r < s do
        x ← x * x mod n
        if x == n − 1 then
            return False
        end if
        r ← r + 1
    end while
    return True
end function
```

The implementation of the Miller-Rabin primality test takes in an input N. To determine the values of s and d for which $2^s \cdot d = N - 1$ and d is odd, s is initialized with a value of 0 and d with a value of $N - 1$. The program utilizes bit manipulation running a while-loop under the condition that the last bit of the binary representation of d is 0, indicating that d is still even. Within this while-loop, the program shifts the bits of d down by 1 to divide by 2 and increments s by 1.

After computing the values of s and d, the program enters a for-loop that iterates $k = 5$ times. For each iteration, the algorithm generates a random number a between 2 and $N - 1$ inclusive. The algorithm then inputs N, a, d, and s into the helper function described previously and checks the output. If the helper function returns true, then N must be composite. In this case the program will terminate early and return false to indicate such. If the helper function returns false, then there is a possibility that N is prime. Under this condition, the program continues on to the next iteration in the for-loop. If after k iterations through the for-loop the program has not terminated, the program returns true to indicate that N is probably prime.

Figure 5. Pseudocode of Miller-Rabin Primality Test

```
function isPrimeMillerRabin(n)
    s ← 0
    d ← n − 1
    while d&1 == 0 do
        d ← d >> 1
        s ← s + 1
    end while
    i ← 0
    while i < 5 do
        a ← rand(2, n − 1) # random number between 2 and n-1 inclusive
        if check_composite(n, a, d, s) then
            return False
        end if
        i ← i + 1
    end while
    return True
end function
```

Summary of Programming Challenge

The programming challenge for this topic provides students with two numbers made up of the same number of digits and asks them to find a path between those two numbers. Every number in the path must be off from the previous number by only one digit, and every number in the path must be prime, including the provided start and end numbers.

The challenge plans to test students with large primes–up to 10-digits long–forcing students to find an efficient way to check the primality of a number as well as an efficient way to branch and explore all possible paths until a successful path is uncovered.

The basic topic of determining the primality of a number is included in this project when students must check possible next numbers in the path to see which are valid. The key twist that this challenge brings is how exactly students find that path. For inputs with large numbers of digits, creating a tree that spans into all possible paths starting from the source number will be expensive for both time and memory. Students will need to implement an efficient algorithm both for determining primality and for finding the shortest path, making this challenge more interesting.

Key Ideas for Solving Programming Challenge

For checking the primality of a number, students must use an efficient and accurate method that can handle larger numbers under tight time constraints. Trial division will take too long under larger inputs, and Fermat's test may not always yield correct results, potentially providing an incorrect path that includes composite numbers. Because of this, students will need to implement a deterministic version of Miller-Rabin test, by using a specific set of bases

instead of randomly chosen bases, allowing any 64-bit integer to be definitively checked in a reasonable amount of time.

In addition to determining the primality of numbers efficiently, students must also find the shortest path between the source and target nodes with accuracy and efficiency. Simple DFS and BFS approaches will not be enough for time constraints when it comes to large inputs. Students will instead have to implement a bidirectional search to limit the space and time used by the algorithm. With this algorithm, students will perform two searches: each starting from one of the two inputted numbers. These each perform a BFS search until they reach a number that the other search has visited in its own BFS tree. When this connection between the two search trees is found, students can combine the paths to that shared number to find the overall path between the source and target numbers.

Students should have learned about DFS and BFS trees before, but may have to challenge their thinking when coming up with a more efficient way to implement these. The bidirectional search still uses BFS trees, so this should not be too difficult for students, but adds an extra twist of complexity.

The time and space complexity for a bidirectional search is $O(b^{d/2})$ where b is the branching factor and d is the shortest distance between the start and end nodes. For each node in the bidirectional search tree, the primality of the number represented by that node must be checked using the Miller-Rabin primality test, which has a time complexity of $O(k \log^3 n)$ where k is the number of trials and n is the number to test. This gives the model solution an overall worst case time complexity of $O(b^{d/2} k \log^3 n)$. The solution will likely run faster as it is able to prune the tree along the way and eliminate branches that contain composite numbers.

Conclusion

This project explores the problem of efficiently determining the primality of a given number and applying that ability to solve a path-finding problem under strict time constraints. This document examines three distinct methods of primality testing, noting the challenges that accompany each. The first of these was trial division, which produces accurate results but has high computational cost when applied to large numbers. This limitation brought us to introduce two probabilistic methods–the Fermat primality test and the Miller-Rabin primality test–emphasizing how each relies on number theoretic properties to reduce runtime while maintaining high accuracy. The Miller-Rabin test provides a practical balance between efficiency and reliability, making it suitable for large inputs.

In the outline for the proposed programming challenge, the section discusses how primality testing alone is not sufficient for the full problem, and extends these ideas into a graph-based problem. Here, each node represents a number and edges are defined by single-digit transformations. To efficiently discover a path, the solution introduces a bidirectional search methodology, significantly reducing the space and time complexities of the program compared to traditional DFS or BFS approaches. By combining efficient primality checking with an optimized search strategy, the final solution is able to handle large inputs while maintaining a high accuracy under tight time constraints.

Overall, this project demonstrates how number theory can be applied to efficiently determine primality and how this can be combined with graph algorithms to solve complex

search problems in a reasonable manner. It also stresses the importance of balancing accuracy and runtime optimizations when approaching large-scale computational problems.